Stage 1 addendum: corrections and added evidence

Calima Data · Search Engine Project · Big Data · ULPGC

This addendum answers the instructor's corrections to the Stage 1 report. Every number comes from a committed script, and the raw files are in `results/stage1/`. The original report is not stored in the repository, so this document complements it and does not replace it.

Table of contents

1. Status of each correction 3
2. Host and software 3
3. How to reproduce everything 4
4. Pipeline corrections in the code 4
5. Missing lookups and new-book discovery 4
6. Storage overhead of the layouts 5
7. Queries across the index structures 5
8. RAM and disk of each index 6
9. Write and update cost as size grows 6
10. Recovery after an interrupted ingestion 7
11. Consistency across structures 7
12. Reproducing the two charts 7
13. Conclusions that changed 8

1. Status of each correction

Correction 11 reports a defect, and Stage 2 depends on what it covers, so it was treated as mandatory. The instructor presents the others as refinements of the report and the code, and all were applied.

#	Correction	Type	Status	Where
1	Table of contents with page numbers	Refinement	Done for this addendum and the Stage 2 report. The original PDF is not available to edit	This section; `stage2/docs/REPORT.pdf`
2	Missing lookup and new-book discovery, per layout	Refinement	Done	Section 5
3	File, directory, auxiliary-state and byte overhead	Refinement	Done. Allocated bytes are an estimate	Section 6
4	Declared query set, latency, same IDs from all structures	Refinement	Done	Section 7
5	RAM and disk of each index	Refinement	Done	Section 8
6	Several sizes, with a fixed operation and batch	Refinement	Done	Section 9
7	Interrupt ingestion, resume, check IDs and pairs	Refinement	Done	Section 10
8	Checks across structures; reapply an update	Refinement	Done. It found a real defect	Section 11
9	CPU, RAM, storage, OS and actual runtime versions	Refinement	Done	Section 2
10	Commands, parameters, revision and raw files of the two charts	Refinement	Regenerated. The original raw files no longer exist	Section 12
11	Lookup missing, no pending-first coordinator, books marked before persisting	**Defect**	Fixed	Section 4
12	Duplicated control and path logic, hard-coded book lists	Refinement	Fixed	Section 4

2. Host and software

All measurements ran on one machine, recorded in `results/stage1/environment.json`.

Item	Value
CPU	AMD Ryzen 7 4800H, 8 physical cores, 16 logical, 2.9 GHz base
RAM	16 GB (16 505 966 592 bytes)
Storage	Intel SSDPEKNW512G8L, 512 GB NVMe SSD; NTFS with 4096-byte clusters
Operating system	Windows 11, build 26300
Python	3.13.5, with SQLite library 3.49.1, requests 2.32.4, pytest 9.1.1, pytest-benchmark 5.3.0, psutil 7.2.2
Java **runtime**	OpenJDK 17.0.20.1 (Temurin). `worker_java` *targets* Java 11 bytecode, but this JVM executes it
Java libraries	JMH 1.35, sqlite-jdbc 3.46.1.3; Maven 3.9.6
Node.js	24.15.0

Experiments used a scratch directory outside OneDrive, because sync activity would disturb disk timings. The machine was not isolated from normal background services such as Windows Defender.

3. How to reproduce everything

```
cd worker_python
python -m pytest test_pipeline.py -q                                          # functional checks
python stage1_experiments.py all --workdir <folder outside OneDrive>       # sections 5 to 11, about 45 min
python stage1_reproduce_charts.py run --workdir <folder outside OneDrive>  # section 12, about 15 min
```

The experiments ran at commit `fac36f2`. The reports flag `dirty: true` because one file was untracked at the time, `stage1_reproduce_charts.py`, which the experiments do not use.

The corpus is declared in `config/stage1_corpus_ids.txt`: 12 real Gutenberg books. The query set is declared in `config/stage1_queries.json`.

4. Pipeline corrections in the code

- **Marking order.** `datamart_index.py` and `datamart_index_sqlite.py` marked a book as indexed before saving or committing, so a crash left a marked book missing from the index. Now the data is persisted first. A crash between the two steps repeats the book, and the indexes ignore duplicates.
- **Atomic writes.** Files are written through a temporary name and renamed. A crash can no longer truncate a term file and lose other books' postings.
- **Readiness.** Metadata and indexing read only books recorded as downloaded, never leftovers of an interrupted download.
- **Time-based retries.** A retry reuses the folder of the previous attempt. Without this, a retry in a later hour would leave a second copy of the book.
- **Lookup.** `lookup.py` answers term-to-postings queries on all three structures.
- **Coordinator.** `coordinator.py` finishes downloaded books that lack indexing before it downloads new ones.
- **Duplication.** `pipeline_common.py` holds the paths, the control-file logic and the tokenizer.
- **Book lists.** All three languages take IDs from the command line or from `config/book_ids.txt`.

`worker_python/test_pipeline.py` provokes real crashes with `os._exit` at exact lines. All 18 checks pass. 16 of the first 17 failed against the previous code; the other passed only because that code ignored its arguments.

The Java and Node.js workers gained only the configurable book list. They implement ingestion and benchmarks, not the complete pipeline, so the other corrections do not apply to them.

5. Missing lookups and new-book discovery

`results/stage1/discovery.csv`, `discovery_checks.json`. All layouts held the same 1010 synthetic books. Time-based books are spread over 7 days × 24 hours, which simulates a week of ingestion. Each value is a mean over 5 rounds.

Layout	Lookup of a missing book (ms)	Lookup of a present book (ms)	Discover new books by full scan (ms)
time-based	9.84 ± 0.23	3.13 ± 0.06	25.4 ± 1.5
book-based	0.074 ± 0.011	0.088 ± 0.016	92.1 ± 11.0
range-based	0.057 ± 0.004	0.074 ± 0.002	12.0 ± 1.9

Every layout discovered exactly the expected new IDs, 1001 to 1010.

Reading. The time-based layout cannot derive a folder from an ID, so a missing book forces a walk over every hour folder: 133 times slower than book-based, and the cost grows with the number of folders. The other two compute the path. A full scan is a different case: book-based is the slowest, because it opens one directory per book, and range-based is the fastest. This is why Stage 2 pairs the time-based layout with a ledger.

6. Storage overhead of the layouts

`results/stage1/storage_layouts.csv`. Allocated bytes are **estimated** by rounding each file up to a 4096-byte cluster. NTFS can store very small files inside the file table, so this is an upper bound for them. The space of the directories themselves is not counted.

Corpus	Layout	Files	Directories	Payload bytes	Allocated bytes (est.)	Overhead (est.)	Auxiliary state
12 real books	time-based	24	2	8 208 375	8 278 016	69 641	1 file, 62 bytes
12 real books	book-based	24	13	8 208 375	8 278 016	69 641	1 file, 62 bytes
12 real books	range-based	24	4	8 208 375	8 278 016	69 641	1 file, 62 bytes
1000 synthetic	time-based	2000	175	1 359 000	8 192 000	6 833 000	none
1000 synthetic	book-based	2000	1001	1 359 000	8 192 000	6 833 000	none
1000 synthetic	range-based	2000	1	1 359 000	8 192 000	6 833 000	none

Reading. The layouts store the same files, so they differ only in directories. At 1000 books, book-based needs 1001 directories and range-based needs one. Rounding matters only for small files: 0.8% on real books, but about five times the payload on the small synthetic ones.

7. Queries across the index structures

`results/stage1/query_latency.csv`, `query_answers.json`. 11 declared terms on the 12 real books, with expected IDs computed independently from the book bodies. **All three structures returned the expected IDs for all terms**, including the absent term `zzzznotaword`.

Latency by structure, as the range of per-term means:

Structure	Mean latency per query (ms)	Repeats per term	What one query does
JSON, reloaded per query	44.7 to 47.5	5	Parses the whole 2.2 MB file
JSON, already in memory	0.0003 to 0.0005	100	One dictionary lookup
Folder hierarchy	0.05 to 0.28	100	Opens one small file; 0.05 when the term is absent
SQLite	0.44 to 0.51	100	Opens a connection and reads the index

Reading. For a single query from a cold process, folders win, because the file name is the index. JSON is about 100 times slower than SQLite unless a long-lived process keeps it in memory, and then it costs the RAM shown in section 8. These SQLite figures include opening a connection on each query, which a service would avoid.

8. RAM and disk of each index

`results/stage1/index_footprint.csv`. RAM is measured in a fresh process for each structure, 3 repetitions: resident memory grown by answering the queries, and the peak working set after an update. An empty Python process with these libraries uses about 27 MB.

Dataset	Structure	Files	Logical size	Allocated (est.)	RAM growth to answer	Peak working set
Real corpus, 51 469 terms	JSON	1	2.09 MB	2.09 MB	11.84 MB	46.4 MB
Real corpus, 51 469 terms	SQLite	1	3.81 MB	3.81 MB	0.67 MB	28.0 MB
Real corpus, 51 469 terms	Folders	51 469	0.52 MB	201 MB	0.04 MB	27.1 MB
Synthetic, 100 000 terms	JSON	1	3.14 MB	3.14 MB	18.93 MB	56.0 MB
Synthetic, 100 000 terms	SQLite	1	4.09 MB	4.09 MB	0.61 MB	27.8 MB
Synthetic, 100 000 terms	Folders	100 000	0.29 MB	391 MB	0.04 MB	27.2 MB

Reading. The roughly 60 KB index of the original report said nothing about this. Folders hold the least data, but one file per term costs the most on disk: the estimate is about 200 MB for a 2 MB index. Because NTFS stores tiny files in its 1 KB file records, the true value should be lower, though still far above the data itself. JSON has the highest RAM use, because it must load everything to answer anything. SQLite barely grows in memory, and its file is about twice the JSON file because of its index pages.

9. Write and update cost as size grows

`results/stage1/scaling.csv`, `scaling_params.json`, charts `results/stage1/charts/scaling_*.png`. The operation is fixed while the starting size grows, with 1 warm-up and 5 measured rounds, and the state restored between rounds.

Adding the same 100 books to a datalake that already holds N books (ms):

Existing books	time-based	book-based	range-based
0	318 ± 27	287 ± 11	268 ± 11
1000	260 ± 18	288 ± 33	315 ± 23
5000	272 ± 9	333 ± 10	325 ± 6
10 000	322 ± 6	329 ± 7	320 ± 7

Adding the same 500-term book (450 existing terms, 50 new) to an index of N terms (ms):

Existing terms	SQLite	JSON	Folders
5 000	8.2 ± 0.8	14.1 ± 0.9	948 ± 17
20 000	11.8 ± 0.6	28.9 ± 3.4	957 ± 15
100 000	27.6 ± 2.1	129 ± 8	956 ± 14

Reading. Datalake writes do not depend on its size up to 10 000 books; differences between layouts are within noise. For the index, a 20-fold size increase gives: folders, flat, but about 950 ms because 500 files are rewritten; JSON, 9.1 times slower, because it rewrites the whole file; SQLite, 3.3 times slower, most likely because the random terms touch more distinct pages. JSON stays between the other two at every size here, and its gap to SQLite widens from 1.7 to 4.7 times.

Folder times are higher than in the original report because term files are now written atomically.

10. Recovery after an interrupted ingestion

`results/stage1/recovery.csv`. This completes the experiment left pending on page 5 of the report. The ingestion of the 12 real books is killed with `os._exit` at a crash point, and then resumed. The resume runs with a clock set to another hour, the worst case for the time-based layout.

Crash point	Marked after the crash	Resume exit code	Marked after resuming	Complete pairs, each layout	Duplicates or orphans	Final IDs match
Between header and body	0	0	12	12	0	Yes
After the files, before marking	0	0	12	12	0	Yes

11. Consistency across structures

`results/stage1/consistency.json`. On the 12 real books:

Check	Result
Metadata against each header	12 books, 0 differences
JSON equals folders, and JSON equals SQLite (every term and posting)	Yes, 51 469 terms and 102 612 postings
Equal to the expected index computed separately from the bodies	Yes
Running the pipeline again changes the state (SHA-256 of metadata and postings)	No
Clearing the tracking files and reindexing changes the state	No
Duplicates in the download control file	None

A defect this check found. The first run reported that all three indexes agreed with each other but not with the book bodies. The rule `\b[a-z0-9]+\b` silently dropped whole words: `español` gives no term at all, because `ñ` counts as a word character and removes the `\b` boundary, and `_italics_`, Gutenberg's emphasis markup, is lost for the same reason. The index had 45 630 terms and now has 51 469 on the same books, 12.8% more. Comparing the structures only with each other could never have revealed this.

12. Reproducing the two charts

`worker_python/stage1_reproduce_charts.py` exports a revision with `git archive`, runs the three suites there with exact commands, stores the raw output, and draws the charts only from those files.

The raw files behind the original charts no longer exist, so the original values cannot be checked. The charts were measured again at the revision that the report describes:

Item	Value
Revision	`b2224d7b489a9cfc33efdf9b5774e73a44b40dd5`, exported with `git archive`
Python	`python -m pytest test_benchmark.py test_index_benchmark.py -q -p no:cacheprovider --benchmark-json=<repo>\results\stage1\charts_raw\python_pytest_benchmark.json`
Java	`mvn -q -B test-compile exec:exec "-Dexec.args=-classpath %classpath org.openjdk.jmh.Main -rf csv -rff <export>\java_jmh.csv"`

Item	Value
Node.js	`node datalakeBenchmark.js` and `node indexBenchmark.js`
Write chart	Ingest 1000 synthetic books into an empty datalake. Python and Node.js: 2 warm-up and 5 measured rounds. Java: per its JMH annotations
Update chart	Add one 500-term book (450 existing terms, 50 new) to a 5000-term index. Python and Node.js: 3 warm-up and 8 measured rounds. Java: per its JMH annotations
Raw files	`results/stage1/charts_raw/`: `manifest.json`, `python_pytest_benchmark.json`, `java_jmh.csv`, `node_*.txt`, `chart_data.csv`
Charts	`results/stage1/charts/write_time.png`, `update_time.png`

Measured values, in ms. Python and Node.js show the standard deviation; Java shows JMH's 99.9% confidence half-width, which is a different quantity, so errors should not be compared across languages. The values the README reported are in brackets.

Language	time-based	book-based	range-based
Python	1945 ± 22 (1888)	2080 ± 34 (2103)	1921 ± 28 (1908)
Java	1578 ± 121 (1277)	1562 ± 82 (1765)	1582 ± 115 (1742)
Node.js	2412 ± 47 (2342)	2559 ± 29 (2389)	2383 ± 20 (2431)

Language	SQLite	JSON	Folders
Python	9.7 ± 0.9 (9.8)	28.7 ± 1.1 (38.7)	596 ± 22 (459)
Java	8.1 ± 0.4 (9.9)	2.5 ± 0.2 (14.1)	442 ± 26 (310)
Node.js	8.4 ± 0.7 (9.4)	5.6 ± 0.5 (6.8)	828 ± 19 (623)

Reading. The rankings that matter survive. Folders are by far the slowest index, now 55 to 98 times slower than SQLite, and SQLite's cost is similar across languages. Two statements do not survive:

- *Java's time-based layout being clearly the fastest.* It is now 1578 ms against 1562 and 1582, with overlapping intervals. No layout is measurably faster in Java. The original 1277 ms is not reproduced.
- *Java's JSON index being on a par with SQLite.* It is now 3.2 times faster than SQLite, and 5.7 times faster than the original 14.1 ms. This is the largest difference. Without the original raw files, its cause cannot be established.

Folder times are also higher than in the README. This run exported the code from before the atomic writes mentioned in section 9, so those writes do not explain the difference. It is not known which machine state or exact code produced the README's values, so the cause stays open, and comparisons across runs of this benchmark should be treated with caution.

13. Conclusions that changed

- The README said SQLite's update cost does not grow with index size. **It does**, 3.3 times over a 20-fold growth, though much less than JSON's 9.1 times. The README has been corrected.
- `time_based` "costs nothing at ingest" remains true. But it does cost at lookup, by two orders of magnitude, which the original evidence could not show.
- Folders are the fastest index for a cold single query and the slowest to update, and they have the largest disk footprint. Ranking structures on writes alone hid that trade-off.

- SQLite remains the best overall choice for the datamart, now on broader evidence, and Stage 2 builds on it.